

Ping Distributed Chat Application – Load and Stress Test Report

Introduction

This report summarizes the performance, bottlenecks, and optimization journey of the *Ping* real-time chat application. Over distributed infrastructure, the application successfully handled 8,000 active virtual users (VUs) communicating concurrently.

Here is the updated report with the new **Architecture Context** section included, and the **Bottleneck Investigation** section restructured to reveal the confirmed culprit first before walking through the debunked suspects.

Architecture Context

(Note: This is a concise version of the full project README.md available on GitHub.)

Ping is a distributed, real-time microservices architecture to handle high-concurrency chatting. The system is divided into specific components:

- **The Services:**
 - **Main Service (ping-main):** A Python (FastAPI) service that acts as the core mediator. It handles standard HTTP API requests, authentication, and all direct database queries.
 - **Connection Manager (ping-cm):** A dedicated Python service handling only live WebSocket connections. It intentionally avoids direct database pushes.
- **The Redis Stack:** Because the system is distributed, Redis helps in finding the worker/replica is in connection with the end user to deliver message.
- **Database:** GCP managed PostgreSQL, mediated by PgBouncer to manage achieve database connection pooling.
- **Infrastructure:** Used Docker Swarm over Google Cloud Platform (GCP). Nginx acts as the static frontend server and also load balancer (with a python service updating the list of replicas available), routing traffic to 4 deployed replicas of the Main Service and 4 replicas of the Connection Manager.

Infrastructure & Deployment Setup

Due to a max limit quota of 6 CPU cores on GCP, the infrastructure was optimally split as follows:

- **Main Service Node:** 3 CPU cores.
- **Worker/Connection Manager Node:** 2 CPU cores.
- **Load Generator (k6):** 1 CPU core.
- **Deployment:** Docker Swarm is used to manage the worker instances, deploying 4 replicas each for the ping-main and ping-cm (Connection Manager) services.

- **Managed Services:** GCP Redis and PostgreSQL.

Configuration blockades at high concurrency

All software come with some default configs. When it comes to high concurrency, these act as blockades. And it requires extensive, repetitive debugging sessions and configuration tunings across the stack. Key adjustments include:

- **OS & Docker:** Increased file descriptor limits (`ulimit -n`) across all Docker containers to prevent socket exhaustion.
- **Database:** Scaled PgBouncer database connections and max client connection numbers to act as a buffer for PostgreSQL.
- **Reverse Proxy:** Updated Nginx's configuration to increase `worker_connections` and `worker_processes`, bypassing the default 1024 connection bottleneck.
- **Hardware:** Increased machine RAM sizes to handle thousands of active TCP/WebSocket connections.

Test Results

I present two primary tests (I have made around 10-12 tests) to evaluate the system:

1. **Load Test (6,000 VUs):** The system operated smoothly with excellent response times. The 95th percentile (P95) latency was exceptionally low at **198 ms**.
2. **Stress Test (8,000 VUs):** The system remained functional but experienced significant performance degradation, pushing the P95 latency to **11 seconds**.

Metric	6k Load Test	8k Stress Test
Peak VUs	6,000	8,000
Test Duration	~11 min	~25 min
HTTP Request Rate (peak)	~200/s	~400/s
HTTP Req Duration — avg	60ms	2s
HTTP Req Duration — P95	174ms	11s
HTTP Req Duration — P99	682ms	14s
HTTP Req Failed Rate	0%	0%
WS Connect Duration — avg	33s	47s
WS Session Duration — P95	~1 min	~1 min
WS Msg Transfer Rate (peak)	~800/s	~750/s
Iteration Duration — avg	~1 min	~1 min

Bottleneck Investigation (Analysing the 8k VU Stress Test)

To understand why latency spiked at 8,000 users, I utilized logs and telemetry to track exactly where the requests were getting delayed. Basically, at what location/point are requests sitting and waiting:

- **Suspect A: Nginx Bottleneck (Debunked):** I suspected the reverse proxy might be failing to route traffic fast enough during this high concurrency.
But with `worker_processes` set to auto and limits raised, Nginx can comfortably handle 40,000 to 50,000 concurrent connections. So, Nginx was not the bottleneck.

- **Suspect B: PgBouncer Queue Bottleneck (Debunked):** Maybe the number of DB connections are low, due to which the DB queries/Client connections are waiting in the PgBouncer .
But, GCP metrics indicated that the average query execution time was around 0.1 to 0.2 ms. Even allowing for 20 ms of network latency, 80 database connections could theoretically process up to 4,000 queries per second. Considering user interaction sleep times and the load split, PgBouncer was receiving a maximum of 2,000–3,000 queries per second, well within the limits.
- **Suspect C: Backend Worker Rejection (Debunked):** Maybe the Python workers were outright refusing to accept the incoming connections from Nginx.
So, I analyzed 0.5% of Nginx's access logs and found that `request_time` and `upstream_response_time` were nearly identical. This proved Nginx wasn't holding packets; it successfully established connections to the Python workers, and the workers accepted them immediately. Thus, debunked.
- **Suspect D: Uvicorn Queue / Backend Replicas (Confirmed):** This was the first and the simple suspect, but got it confirmed by brute forcing and eliminating all other suspects using logs and telemetry.
The root cause of the delay was found within the Python worker's Uvicorn queue. At high concurrency, the high volume of incoming requests exceeded the processing rate of the workers. Because Python is limited by the Global Interpreter Lock (GIL), the requests piled up inside the Uvicorn application queue, causing the massive 11-second latency.

Conclusion & Future Architecture Improvements

With the current 5-core backend limit, the Python-based architecture handles upto 8,000 active users. To handle higher concurrency, two architectural shifts are required:

1. **Rewrite Connection Manager in Go:** I intentionally decoupled the connection manager service from database writes (the Main service acts as mediator for DB requests). Rewriting the Connection Manager in Golang will drastically increase capacity. Go uses lightweight goroutines, acting like an "octopus" that dynamically and efficiently distributes concurrent WebSocket connections across all available CPU cores.
2. **Scale Workers Horizontally (Docker Swarm Replicas/ K8s Pods):** Scaling workers horizontally includes both vertical scaling and horizontal scaling of the virtual machine. Because a standard Python worker is single-threaded due to the GIL, and my gRPC server is tightly coupled with Uvicorn (preventing the use of Gunicorn process managers), the fix is to increase the number of replica containers in Docker Swarm to spread the load across more Uvicorn instances.

Scope for Application & Testing Improvements

Beyond infrastructure, future updates will focus on software robustness:

- Investigating and fixing "missed message" issues.
- Optimizing message delivery latency when both users are actively connected.
- Writing robust integration tests to verify database persistence (saving all the messages sent during the test and refreshing the page to check if all messages actually saved).
- Deepening load test scenarios specifically for high-traffic Group Chats.